

Once4All: Skeleton-Guided SMT Solver Fuzzing with LLM-Synthesized Generators

Maolin Sun
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
merlin@smail.nju.edu.cn

Yibiao Yang*
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
yangyibiao@nju.edu.cn

Yuming Zhou
State Key Laboratory for Novel Software Technology, Nanjing University
Nanjing, China
zhouyuming@nju.edu.cn

Abstract

Satisfiability Modulo Theory (SMT) solvers are foundational to modern systems and programming languages research, providing the foundation for tasks like symbolic execution and automated verification. Because these solvers sit on the critical path, their correctness is essential, and high-quality test formulas are key to uncovering bugs. However, while prior testing techniques performed well on earlier solver versions, they struggle to keep pace with rapidly evolving features. Recent approaches based on Large Language Models (LLMs) show promise in exploring advanced solver capabilities, but two obstacles remain: nearly half of the generated formulas are syntactically invalid, and iterative interactions with LLMs introduce substantial computational overhead. In this study, we present Once4All, a novel LLM-assisted fuzzing framework that addresses both issues by shifting from direct formula generation to the synthesis of generators for reusable terms (i.e., logical expressions) . Specifically, Once4All uses LLMs to (1) automatically extract context-free grammars (CFGs) for SMT theories, including solver-specific extensions, from documentation, and (2) synthesize composable Boolean term generators that adhere to these grammars. During fuzzing, Once4All populates structural skeletons derived from existing formulas with the terms iteratively produced by the LLM-synthesized generators. This design ensures syntactic validity while promoting semantic diversity. Notably, Once4All requires only **one-time LLM interaction investment**, dramatically reducing runtime cost. We evaluated Once4All on two leading SMT solvers: Z3 and cvc5. Our experiments show that Once4All has identified 43 confirmed bugs, 40 of which have already been fixed by developers.

CCS Concepts: • **Software and its engineering → Formal methods**; **Software testing and debugging**.

Keywords: SMT solver, Fuzzing, Large Language Models

ACM Reference Format:
Maolin Sun, Yibiao Yang, and Yuming Zhou. 2026. Once4All: Skeleton-Guided SMT Solver Fuzzing with LLM-Synthesized Generators. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, Article 111, 17 pages. https://doi.org/10.1145/3779212.3790195

1 Introduction

Satisfiability Modulo Theory (SMT) solvers are essential systems for automatically determining the satisfiability of logical formulas with respect to background theories, such as arithmetic, bit-vectors, and strings [4]. They play a critical role across multiple domains, including computer systems [16, 17, 19, 31], computer networks [14, 15, 20, 28, 41], and programming languages [21, 26, 27]. For instance, the renowned symbolic execution engine KLEE [6] relies on SMT solvers to check the feasibility of program paths. However, bugs hidden in SMT solvers can mislead client applications, resulting in serious consequences and even security vulnerabilities [18]. Thus, ensuring the reliability of SMT solvers is of critical importance.

To this end, various approaches have been proposed for testing SMT solvers, which can be broadly classified into three categories: (1) *generation-based* [5, 32, 48], (2) *mutation-based* [22, 29, 35, 40, 49, 50], and (3) *LLM-based* [39, 51]. Generation-based approaches typically generate formulas from scratch based on predefined generation rules or grammar. For instance, ET [48] is a representative technique that enumerates formulas based on grammar. Mutation-based approaches, on the other hand, involve mutating existing formulas to generate new ones. One of the most advanced approaches in this category is HistFuzz [40], which utilizes

*Corresponding author.

Authors' Contact Information: Maolin Sun, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, merlin@smail.nju.edu.cn; Yibiao Yang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, yangyibiao@nju.edu.cn; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China, zhouyuming@nju.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03
https://doi.org/10.1145/3779212.3790195

historical bug-triggering formulas to generate new formulas. Furthermore, researchers have explored LLM-based approaches inspired by the advancement of large language models (LLMs). These approaches aim to generate SMT formulas directly using LLMs [39, 51]. For instance, LaST [39] retrained an LLM, aiming to generate effective test formulas from scratch. Fuzz4All [51] is a universal framework that is applicable to various software systems, including SMT solvers. It can generate test inputs for SMT solvers by prompting the generation LLM with the prompt generated by another LLM.

Despite notable progress, existing SMT solver fuzzing approaches still face several challenges. Generation-based techniques often struggle to produce diverse formulas because they rely on rigid generative rules, and their from-scratch generation process makes it costly to reach deeper solver states. In comparison, mutation-based approaches can leverage seed formulas to more efficiently explore deeper execution paths. However, their effectiveness still depends heavily on the design of mutation strategies [35, 39]. For instance, OpFuzz [49] only mutates the operators in seed formulas, which may restrict the diversity of generated formulas. More recently, LLM-based techniques directly prompt large language models to generate complete SMT formulas. While attractive in principle, such approaches often produce inputs that are syntactically or semantically invalid (approximately 50%) [39, 51], sometimes lead to nonsensical bug reports[1], and incur substantial computational cost due to heavy interaction with LLMs. Meanwhile, the SMT-LIB standard [2], the primary input language for SMT solvers, continues to evolve. The recent release of version 2.7 and the planned introduction of SMT-LIBv3 reflect the community's ongoing efforts to support richer language features. Concurrently, SMT solvers like cvc5 have introduced extended, solver-specific theories [10] to better express real-world constraints. Collectively, these developments increase the demand for fuzzing techniques that can adapt to evolving changes for better testing emerging solver capabilities.

Our Approach. In this study, we propose Once4All, a novel SMT solver fuzzing framework that unifies the strengths of mutation-based fuzzing with the advanced capabilities of LLMs. While mutation-based fuzzing techniques achieve high testing throughput and deep state exploration by mutating existing test cases, adapting them to support newly introduced or evolving SMT features typically requires substantial manual effort. Once4All overcomes this limitation by leveraging LLMs' contextual understanding and program synthesis abilities to enable rapid, automated adaptation to changes in solver input language. Specifically, Once4All includes two key phases: (1) ***LLM-assisted generator construction*** and (2) ***skeleton-guided mutation***. In the first phase, Once4All prompts LLMs to extract and summarize the context-free grammars (CFGs) of SMT-LIB theories, drawing from the official documentation [2]. We also include grammar for solver-specific extensions, such as the Bag and Set theories in cvc5 [10]. Based on these CFGs, LLMs then synthesize reusable generators that emit Boolean terms (i.e., logical expressions) conforming to the theories. This process is a *one-time investment*, only requiring repetition when grammar evolves. Note that this phase is essential because no existing resource provides off-the-shelf grammars or generators that fully cover both the latest SMT-LIB standard and diverse solver-specific extensions [48]. Particularly, many extensions are only informally documented [10], making manual construction difficult, whereas LLMs can readily interpret and adapt to such documentation. The second phase employs these generators within a skeleton-guided mutation procedure. The skeletons are derived by removing small Boolean terms from seed formulas [40]. The terms generated by the LLM-implemented generators are then synthesized into the skeletons, producing novel test formulas. Skeletons help target deeper solver logic while also complementing gaps in SMT-LIB documentation. For example, important symbols like quantifiers are not specified in theory documentation, limiting what a purely grammar-based generator can produce. By utilizing existing skeletons, Once4All expands the expressiveness of generated formulas. Once4All finally performs differential testing across multiple solvers to detect behavioral divergences indicative of bugs.

We have implemented Once4All as a practical and extensible fuzzing tool for SMT solvers. To evaluate the effectiveness of Once4All, we applied it to the leading SMT solvers, i.e., Z3 [11] and cvc5. Ultimately, Once4All has successfully identified 45 real bugs in Z3 and cvc5, of which 43 were confirmed, and 40 of those confirmed bugs were fixed by the developers. Notably, Once4All exhibits effectiveness in identifying bugs across various theories, including the recently introduced theories and solver-specific extensions. Besides, our experiments also exhibit that Once4All outperforms advanced SMT solver fuzzers in terms of achieved code coverage and effectiveness.

Contributions. We make the following major contributions:

- ⋆ ***Novelty:*** We propose a novel approach for fuzzing SMT solvers, combining the strengths of mutation-based fuzzing with the capabilities of LLMs. By strategically using LLMs, we enhance the generation of test formulas, allowing our approach to adapt more effectively to the evolving features of SMT solvers.
- ⋆ ***Significance:*** Our work focuses on SMT solvers, which are fundamental components in formal verification and program analysis. Improving their robustness is of paramount importance. Furthermore, we present a fresh perspective on applying LLMs in software testing, a paradigm that can be extended to other software systems beyond SMT solvers.

[1] https://github.com/Z3Prover/z3/issues/6818

- ⋆ ***Practicality:*** We implement our idea as a practical tool Once4All. Once4All has led to 45 bug reports for the leading SMT solvers Z3 and cvc5, of which 43 are confirmed and 40 are fixed by developers. Experimental results also demonstrate that Once4All outperforms the state-of-the-art SMT solver fuzzers in terms of achieved code coverage.

Paper Organization. The remainder of this paper is organized as follows. Section 2 provides an overview of the necessary background and motivates our approach. Section 3 formalizes our approach and describes the implementation of Once4All. Next, we elaborate on our extensive evaluation in detail in Section 4, and Section 5 is dedicated to in-depth discussions on the results. Finally, we survey related work in Section 6, and the conclusion is in Section 7.

2 Background and Motivation

In this section, we first overview the SMT-LIB language and the standard for describing SMT formulas. Then, we motivate our technique by illustrating a real-world bug in the cvc5 SMT solver.

2.1 SMT-LIB Language

SMT-LIB serves as the standard input language for SMT solvers, providing a unified framework for expressing logical formulas and background theories. Developed by the SMT-LIB initiative, it defines a formal syntax and semantics for declaring symbols, asserting constraints, and querying satisfiability. The core commands in SMT-LIB include `declare-fun` for symbol declarations, `assert` for adding logical constraints, and `check-sat` for determining satisfiability. Note that variables can be treated as functions with no arguments. For example, the statement `(declare-fun x () Int)` introduces an integer variable `x`, and `(assert (and (< x 3) (> y 1)))` requires that `x` is less than 3 and `y` is greater than 1. The solver returns `sat` if all assertions can be satisfied simultaneously, or `unsat` otherwise. SMT-LIB is continuously evolving to support more complex and real-world scenarios [38]. Recent versions, such as version 2.7 [3] (released in 2025), have introduced advanced features like polymorphic sorts, higher-order functions, and wildcards in pattern matching. These extensions significantly enhance the expressiveness of SMT solver inputs, allowing for the modeling of more intricate problems without compromising backward compatibility or the core principles of the language.

2.2 Motivation

The importance of SMT solvers in the software engineering community has led to the development of numerous fuzzing techniques aimed at ensuring their correctness and robustness. These methods have successfully uncovered many bugs [32, 35, 40, 49]. However, most existing approaches rely on manually designed input-generation strategies. As the SMT-LIB standard evolves and solvers introduce new features, such static strategies often struggle to keep pace. This limitation reflects a broader challenge in software testing: as systems evolve, subtle bugs can emerge that existing methods may overlook, making adaptive testing approaches increasingly necessary [51]. To illustrate, consider a real bug in the cvc5 solver, shown in Figure 1. This bug occurred in cvc5's handling of the sequence theory, which was extended to support comprehensive operations for modeling real-world sequences. In this example, `seq.rev` reverses a sequence, `seq.len` computes its length, and `seq.nth` retrieves an element at a specific position (i.e., *n*-th). The formula in Figure 1 triggered unexpected behavior in cvc5. Specifically, the sequence-related term could not be evaluated to a concrete constant, causing the solver to incorrectly reject a valid model. This example illustrates a broader issue: specialized and newly added features often involve intricate logic that generic fuzzing techniques, originally designed for established theories, fail to test adequately. Fortunately, recent advances in LLMs offer a promising way forward. In particular, LLMs demonstrate strong capabilities in understanding both code and documentation [51], suggesting they could help automate the tracking and testing of newly integrated solver features without heavy human effort.

Observation 1

Beyond established theories, newly introduced extensions and solver-specific features are also sources of bugs and therefore demand targeted testing.

Furthermore, the logical structure of the input formula, i.e., its skeleton, can be critical in exposing such bugs. In the example shown in Figure 1, developers noted that the quantifier was also a critical factor in triggering the bug, even though it was semantically largely irrelevant to the problematic core expression.[2] In other words, the bug cannot be reproduced without the presence of the quantifier `exists ((f Int))`, despite its lack of direct interaction with the assertion's body. Its presence altered the solver's internal execution path, revealing the bug in model evaluation. This demonstrates that seemingly minor structural elements within a formula can significantly influence a solver's behavior and are critical for uncovering bugs, a phenomenon also observed in other studies [40]. Therefore, this leads to the following observation.

Observation 2

The skeleton of the input formula, including quantifiers and logical connectives, can profoundly influence solver behavior and is crucial for revealing bugs.

[2] https://github.com/cvc5/cvc5/pull/11930#discussion_r2104698083

```
(declare-fun s () (Seq Int))
(assert (exists ((f Int))
 (distinct (seq.len (seq.rev s))
 (seq.nth (as seq.empty (Seq Int)) (div 0 0)))))
(check-sat)
```

Figure 1. A bug-revealing formula involving the Seq theory extended in cvc5. #11924

In practice, this bug was discovered by our proposed technique, Once4All. Specifically, Once4All extracts skeletons from existing formulas by removing their atomic subformulas. In this example, the skeleton is `(exists ((f Int)) <placeholder>)`. Subsequently, Once4All fills in the placeholders with newly generated formulas across different theories. We will elaborate on the details in Section 3.

3 Approach

In this section, we provide an overview of our proposed approach, Once4All, followed by a detailed explanation.

3.1 Overview

This study introduces a novel approach called Once4All, illustrated in Figure 2. Once4All's workflow is structured into two primary phases: (1) LLM-assisted generator construction and (2) skeleton-guided mutation. The first phase focuses on building robust test case generators. We begin by gathering comprehensive documentation on various SMT theories from official sources, including the SMT-LIB standard and leading SMT solvers like Z3 and cvc5. Subsequently, we leverage LLMs to automatically summarize the context-free grammar (CFG) for each theory. With these grammars, we then instruct LLMs to implement generators capable of producing Boolean terms for each theory. These generators are designed to create Boolean terms that adhere strictly to their respective CFGs, include necessary declarations such as `declare-fun` and `declare-datatypes`, and are fully compatible with the SMT-LIB specification. This initial phase is a one-time setup process, requiring updates only if new theories or features are introduced to SMT solvers. Upon completion of the generator construction, we advance to the second phase: skeleton-guided mutation. In this phase, Once4All takes existing formulas, such as those found in official SMT-LIB benchmarks, and extracts their skeletons by removing specific Boolean terms. Once4All then calls upon the generators developed in the first phase to produce new Boolean terms. These newly generated terms are then integrated into the extracted skeletons, forming novel test formulas. Finally, Once4All employs differential testing by feeding the generated test formulas to multiple SMT solvers and analyzing their behaviors to identify discrepancies.

Algorithm 1: LLM-Assisted Generator Construction

Input : Set of documentation for SMT theories Doc_T, Large Language Model LM
Output: Set of self-corrected generators G

Procedure GenConstruct(Doc_T, LM)
  $G \leftarrow \emptyset$
  foreach $doc \in Doc_T$ **do**
    /* Extract CFG from documentation */
    $Prompt_1 \leftarrow$ ConstructPrompt (doc)
    $CFG \leftarrow$SummarizeCFG ($Prompt_1$, LM)
    /* Use CFG to construct generator */
    $Prompt_2 \leftarrow$ ConstructPrompt (CFG)
    $g_t \leftarrow$ ConstructPrompt ($Prompt_2$)
    /* Check and correct the generator */
    $valid_g_t \leftarrow$ Correct (g_t, LM, $Solvers$)
    $G \leftarrow G \cup valid_g_t$

Function Correct(g, LM, $Solvers$)
  $iter \leftarrow 0$; $max_iter \leftarrow 10$
  $best_g \leftarrow g$; $max_valid \leftarrow 0$
  $sample_num \leftarrow 20$
  while $max_valid < sample_num$ **and** $iter < max_iter$ **do**
    $Errors \leftarrow \emptyset$
    $iter \leftarrow iter + 1$
    $terms \leftarrow$ Generate (g, $sample_num$)
    $valid_cnt \leftarrow 0$
    foreach *term* $t \in terms$ **do**
      $ParseRes \leftarrow$ Parse (t)
      if CheckErr *(ParseRes)* **then**
        $Errors \leftarrow Errors \cup ParseRes$
      else
        $valid_cnt \leftarrow valid_cnt + 1$
    if $valid_cnt > max_valid$ **then**
      $max_valid \leftarrow valid_cnt$
      $best_g \leftarrow g$
    if $valid_cnt < sample_num$ **then**
      $P \leftarrow$ ConstructPrompt ($Errors$)
      $g \leftarrow$ Refine (P, LM)
  $g \leftarrow best_g$

3.2 LLM-Assisted Generator Construction

Construction of Generators. The initial phase focuses on building generators for various SMT theories, as illustrated in Algorithm 1. This automation is key to ensuring our fuzzing framework can readily adapt to the continuous evolution of SMT solvers. Here, we divide this complex task into two sub-tasks: (1) summarizing context-free grammars and (2) implementing the generators, as shown in Algorithm 1. Specifically, our approach commences by

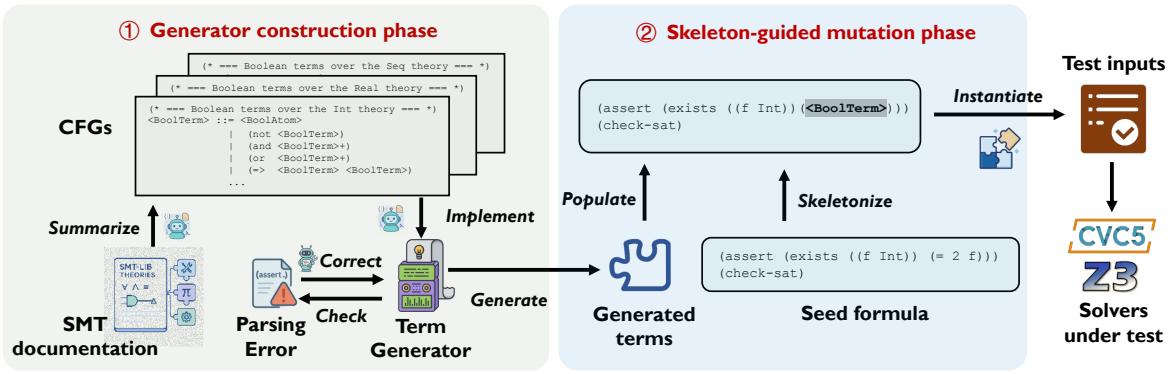


Figure 2. Overview of Once4All.

gathering comprehensive documentation for a range of SMT theories. This includes standard theories defined in SMT-LIB [2] (e.g., `Ints`, `Reals`, `Bit-vectors`) and solver-specific extensions, such as Z3's `Unicode` theory [30] and cvc5's `Set` theory [10]. Notably, many of these extensions are described only informally, making manual grammar construction difficult and prone to inconsistency. Using this collected documentation, Once4All employs LLMs to automatically summarize the context-free grammar (CFG) for each theory. This summarization (Line 5) is guided by a pre-defined prompt template, which is shown in Figure 3a. Subsequently, these extracted CFGs form the basis for implementing the generators (Line 7). Once4All again leverages LLMs, providing them with the CFGs and a detailed instruction as shown in Figure 3b. This step aims to produce generators capable of creating syntactically valid Boolean terms (i.e., logical expressions) for each theory. Specifically, the generated Boolean terms are expected to conform to the respective CFGs, include necessary SMT-LIB declarations (e.g., `declare-fun`, `declare-datatypes`), and adhere to the SMT-LIB specification. This automated pipeline facilitates the creation of generators with a consistent interface, simplifying the future integration of new theories.

Self-correction of Generators. Before these generators are employed in the fuzzing process, we incorporate a self-correction mechanism to ensure the syntactic and semantic validity of the Boolean terms they produce (Line 8). This mechanism is essential because the CFGs extracted from the documentation may not fully capture the intricacies of the SMT-LIB specification, potentially resulting in invalid terms. For example, the CFGs for the BitVector theory may fail to enforce consistent bit-widths for operations that require operands of equal width, such as `bvadd` and `bvmul`. The self-correction process begins by tasking a newly constructed generator with producing a small set of sample terms (Line 17). The number of sample terms is set to 20, as shown in line 13. Each term is then augmented with necessary SMT-LIB commands and parsed by multiple SMT solvers, such as Z3 and cvc5. A term is deemed valid if at least one solver parses it without error (Line 20). If any of the sample terms are found to be invalid, Once4All initiates a refinement loop (Line 14). The parsing errors reported by the solvers are collected and used to construct a new prompt for the LLM based on the devised template (Figure 3c), instructing it to refine the generator's implementation (line 30). Notably, if multiple terms are identified as invalid, we first use LLMs to distill and deduplicate the error messages, and then construct a prompt based on the distilled messages to guide the improvement of the generators. This feedback-driven cycle is repeated until all sample terms are valid or a predefined maximum number of iterations (e.g., 10) is reached. If the latter occurs, the version of the generator that produced the highest number of valid expressions across the iterations is retained (line 31). This self-correction mechanism is designed to improve the quality of the generated terms, thereby enhancing the overall effectiveness of Once4All. With these robust generators, Once4All is well-equipped for the subsequent phase of skeleton-guided fuzzing. Notably, the self-correction mechanism cannot guarantee that the generators produce only well-formed terms; however, a small proportion of ill-formed terms is tolerable in the context of fuzzing.

3.3 Skeleton-guided Mutation

Once4All generates diverse test formulas using a *skeleton-guided mutation* strategy (Algorithm 2). This design choice enables Once4All to explore deeper solver behaviors while overcoming gaps in the SMT-LIB specification. For instance, certain constructs such as the quantifiers `forall` and `exists` are not fully covered by SMT-LIB theory documentation, limiting what a purely grammar-based generator can produce. By leveraging existing formulas as structural templates, Once4All extends the expressiveness of generated inputs and reaches behaviors that grammar-only methods cannot. Specifically, the process begins with two inputs: a collection

Grammar Summarization Prompt

```
### Please generate a context-free grammar (CFG) in
BNF or EBNF format that produces Boolean terms valid
in the SMT-LIB syntax for the [THEORY] theory. The
grammar should accurately reflect the following
theory-specific constructs and constraints:

### Documentation
[OFFICIAL DOC]
```

(a) Prompt template for grammar summarization.

Generator Implementation Prompt

```
Please implement a Python-based random formula
generator for [THEORY] using the provided context-free
grammar. The `generate_[THEORY]_formula_with_decls()`
function should return two strings: symbol
declarations and the formula terms (without commands
like `assert`). …

### Context-free grammar
[CFG]
```

(b) Prompt template for generator implementation.

Self-Correction Prompt

```
The provided Python code for an SMT formula generator
is producing syntactically invalid term and causing
solver errors. Your task is to correct the Python code
to ensure it generates syntactically valid terms.
Focus solely on fixing the errors and improving the
validity of the generated terms. Provide only the
complete, corrected Python implementation, wrapped in
triple backticks (```).

### Invalid terms and the corresponding errors:
[INVALID TERMS WITH ERRORS]

### Current generator implementation
```
[GENERATOR IMPLEMENTATION]
```
```

(c) Prompt template for self-correction of generators.

Figure 3. Prompt template used in LLM-assisted generator construction phase.

of seed formulas, *Seeds*, and a set of term generators, G, obtained from the previous phase. These are combined with lightweight structural synthesis to create a wide variety of formulas that are effective at uncovering solver bugs. In each iteration, Once4All first randomly selects a seed formula f from *Seeds* (Line 4). Once4All then derives the *skeleton* of the formula by randomly removing a subset of its atomic formulas, i.e., basic sub-formulas without logical connectives like and and or, and replacing them with a `<placeholder>` (Line 6). This transformation enables flexible insertion of Boolean terms while preserving the overall structure and

Algorithm 2: Main fuzzing loop of **Once4All**.

Input : Generators G, Seed formulas *Seeds*
Output: Bugs List *Bugs*

```
1  Procedure Fuzz(G, Seeds, Solvers)
2      Bugs ← ∅
3      while no termination by user do
4          f ← RandomSelect(Seeds)
5          repeat
6              skeleton ← Skeletonize(f)
7              g ← RandomSelect(G)
8              terms ← Generate(g, skeleton)
9              f ← Synthesize(terms)
10             Results ← Validate(f, Solvers)
11             if Discrepancy(Results) then
12                 Bugs ← Bugs ∪ {f}
13         until mutation termination criterion is met
```

validity of the formula. Next, Once4All selects a generator $g \in G$ (Line 7) at random and uses it to produce a set of Boolean terms (Line 8). Before inserting these terms into the skeleton, Once4All checks for matching sorts (e.g., , `Bool`, `Int`, or `Real`) between the variables in the generated terms and those in the skeleton. When compatible, variables in the generated terms are randomly replaced with corresponding variables from the skeleton to increase semantic interaction between the inserted content and the original structure.

These modified terms are then integrated into the skeleton, yielding a new formula f (Line 9). The formula is then validated using multiple solver implementations (Line 10). If the solvers produce inconsistent results or one crashes, the formula is recorded in the bug list *Bugs* (Lines 11 - 12). In cases where the formula includes solver-specific features, we compare results across different versions of the same solver. To identify the source of a discrepancy, we use the `get-model` command to extract a model, i.e., a set of variable assignments, from the solver that returns `sat`. We then re-evaluate the formula against all solvers using this model. If the formula definitely evaluates to `sat` with the model, we report a soundness bug in any solver that returned `unsat`, and vice versa. If the model fails to satisfy the formula, we also classify it as an invalid model bug. This mutation-validation process continues until a user-defined stopping condition is met, steadily generating new formulas for bug hunting.

Illustrative Example in Steps. We illustrate the skeleton-guided mutation process with a concrete example, shown in Figure 4. The procedure unfolds in three steps:

Step 1: Skeleton extraction. Once4All first selects a seed formula from the seed pool (Figure 4a). It then abstracts away certain atomic formulas at random, replacing them with `<placeholder>` markers, as depicted in Figure 4b.

```
(declare-fun T () Int)
(assert (or (= T 0) (< T 1)))
(check-sat)
```

(a) Original seed formula.

```
(declare-fun T () Int)
(assert (or <placeholder> <placeholder> ))
```

(b) Skeleton extracted from the seed.

```
((_ divisible 3) (mod int0 3))
(= str0 "")
```

(c) Boolean terms generated from generators.

```
(declare-fun T () Int)
(assert (or ((_ divisible 3) (mod T 3)) (= str1 "")))
(check-sat)
```

(d) Synthesized formula.

Figure 4. Illustrative process of Once4All's skeleton-guided mutation.

This preserves the original logical structure while leaving positions open for new terms.

Step 2: Generator invocation. Once4All then invokes generators from G to produce candidate Boolean terms. Notably, generators from different theories can be used simultaneously. In this example, two terms are produced by the `Int` and `String` generators (Figure 4c), respectively. Before inserting them into the skeleton, Once4All checks sort compatibility with variables from the original seed and adapts variables as needed. For instance, the variable `int0` in the first term depicted in Figure 4c is replaced with the existing integer variable `T` from the seed formula, enhancing semantic interactions in mutants. This adaptation is especially valuable for complex seed formulas, where only a subset of sub-formulas may be replaced.

Step 3: Formula synthesis. Finally, the adapted terms are inserted into the skeleton to produce a new formula (Figure 4d). This synthesized formula is then tested across multiple solvers. Any crash or inconsistency in solver outputs indicates a potential bug.

This example is adapted from a real crash in cvc5#12058 detected by Once4All, which was later fixed by developers.

3.4 Implementation

We implemented Once4All as a practical and extensible fuzzing tool for SMT solvers using Python. Specifically, for the LLM-assisted generator construction phase, we utilize GPT-4 [33] to assist in summarizing SMT grammars, implementing generators, and refining them through self-correction. We selected GPT-4 due to its strong performance on programming tasks and its ability to follow natural language instructions effectively [33]. In the skeleton-guided mutation phase, Once4All applies ten iterations of mutation to each input seed formula, a configuration commonly employed in relevant research practices [35].

4 Experimental Evaluation

This section presents a comprehensive evaluation of the effectiveness of Once4All.

4.1 Evaluation Setup

Research Questions. The experiments conducted aim to answer the following research questions:

- **RQ1:** Can Once4All be used to expose new real bugs in SMT solvers? (Section 4.2)
- **RQ2:** How does Once4All compare with state-of-the-art SMT solver fuzzers? (Section 4.3)
- **RQ3:** How do Once4All's different components contribute to its overall performance? (Section 4.4)

Types of Bugs. Following the practices of previous studies [40, 49, 50, 53], we categorize the bugs found in SMT solvers into three main types. The three categories of bugs are defined as follows:

• *Soundness bugs*: This type of bug occurs when two solvers provide opposite results for the same formula, where one solver reports `sat` while the other reports `unsat`. Such inconsistencies are considered soundness bugs.

• *Invalid model bugs*: An invalid model bug occurs when a solver correctly identifies a formula as satisfiable (`sat`), but the provided model fails to satisfy the constraints in the formula.

• *Crash bugs*: This type of bug occurs when a solver exhibits abnormal behavior during the solving process, such as assertion violations and segmentation faults.

Environment. We conduct all of our experiments on a machine equipped with an Intel Xeon CPU Gold-6248 processor (20 cores and 256GB RAM) running a Docker container with the Ubuntu 20.04 64-bit operating system. In line with prior studies [40, 53], we set the time limit for solving queries for each test formula to 10 seconds.

Test Seeds. Once4All is a general-purpose fuzzer that accepts any existing SMT formula as input. Over the years, the SMT community has accumulated a vast collection of formulas for testing and benchmarking. For instance, in addition to the SMT-LIB official benchmark suite [8], prior work [39, 40] curated approximately 3,700 bug-triggering formulas from issue trackers of major solvers including Z3 and cvc5, and found them to be of higher quality in solver testing. In our experiments, we use the bug-triggering formulas released

by these studies [39, 40] as seed inputs, as they demonstrate high bug-finding potential while keeping the seed set size manageable for fair comparison. To avoid data leakage, we re-execute all seed formulas on the latest solver versions and remove any that still trigger previously reported bugs.

4.2 RQ1: Bug Detection

The objective of *RQ1* is to examine whether Once4All can detect new real bugs in SMT solvers.

4.2.1 Experimental Setup.

Targeted Solvers. We evaluate our approach on Z3 and cvc5, two widely used SMT solvers that support a broad range of SMT-LIB features and have been extensively studied in prior work [35, 40, 49]. Following common practice [35, 40], we primarily test solvers in their default configuration, which reflects standard usage by practitioners. We enable a small set of essential options for detecting invalid models, including `model_validate=true` in Z3 and `-check-models` in cvc5, which are generally considered part of the default mode [49]. We additionally explore selected non-default options recommended by solver developers, such as the cvc5 fuzzing guidelines [9], in our bug-hunting campaign. All experiments are conducted on the latest trunk versions to avoid reporting duplicate bugs.

Bug Inspection and Reduction. We follow a semi-automated workflow to inspect and reduce potential bugs detected by Once4All. Our goal is to avoid redundant reports, minimize manual effort, and provide developers with clear, minimal reproducing cases. For crashes, Once4All automatically clusters failing cases using their crash stacks. We treat all crashes that reach the same code location as a single issue. This grouping requires minimal manual inspection. Exception information reported by solvers is used to further refine de-duplication. For soundness and invalid model issues, we group cases by the theory involved, following prior work [50]. A lightweight script identifies the theory of each formula, enabling us to manually inspect only one representative per group. Once developers provide a fix, all remaining cases in that group are automatically revalidated. Furthermore, to improve developer understanding and ease the patching process, we reduce bug-triggering formulas using delta-debugging tools. We primarily rely on `ddSMT` [23]. For cases not supported by `ddSMT`, such as those involving certain solver-specific theories, we instead apply C-Reduce [36]. The resulting minimized formulas help ensure concise and actionable reports.

4.2.2 Results.

Statistics of Bugs. Once4All generated a large and diverse input set during our bug-hunting campaigns. Across all runs, it produced approximately ten million test cases. On average, each formula is 4,828 bytes, and using these inputs, Once4All identified 727 bug-triggering formulas, which were then triaged and reported to the developers of the respective solvers. Table 1 presents the status of the unique bugs discovered by Once4All in Z3 and cvc5. In total, Once4All has reported 45 bugs across the three solvers, of which 43 bugs have been confirmed by the developers. As of the time of writing, 40 bugs have been fixed, leaving the remaining bugs awaiting processing by the developers. Despite our best efforts to de-duplicate the bugs before reporting them, it is impossible to guarantee the complete absence of duplication. However, the small number of duplicate bugs suggests that the de-duplication process is effective. Notably, none of the bugs reported by Once4All have been marked as *won't fix* or *invalid* by the developers, indicating that the bugs identified by Once4All are valid and that our pre-reporting inspection process is effective.

Table 1. Status of bugs found in the solvers.

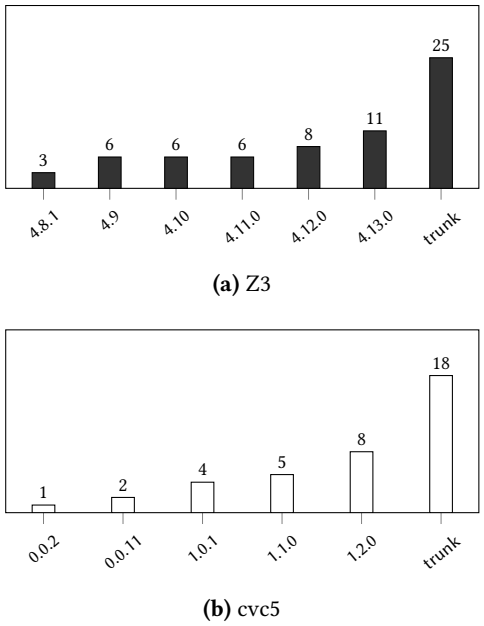
Status	Z3	cvc5	Total
Reported	27	18	45
Confirmed	25	18	43
Fixed	24	16	40
Duplicate	2	0	2

Table 2. Bug types among the reported bugs.

Type	Z3	cvc5	Total
Crash	20	15	35
Invalid model	4	2	6
Soundness	3	1	4

Table 2 depicts the distribution of the bug types of the confirmed bugs. Specifically, the majority of the confirmed bugs (35 out of 43) are crash bugs. The remaining bugs, including six invalid model bugs and four soundness bugs, affect the correctness of the solvers. These results demonstrate that Once4All is capable of exposing real bugs in SMT solvers. Note that among the bugs discovered by Once4All, **11 bugs involve newly added or solver-specific theories** that existing SMT solver fuzzers are fundamentally incapable of uncovering, either by design or in principle. We present concrete examples of these bugs in Section 4.5.

Bug Lifespan. To further understand the bugs unveiled by Once4All, we conducted an analysis of their impact on the release versions of Z3 and cvc5, thereby revealing the bugs' lifespans. To investigate the longevity of bugs, similar to prior work [39], we select Z3 release versions from 4.8.1 to 4.13.0 and cvc5 release versions from 0.0.2 to 1.2.0 as the subjects. Notably, Z3-4.8.1 was released over five years ago, in October 2018, while cvc5-0.0.2 the first official release of cvc5 in October 2021. We evaluated each solver's



(a) Z3

(b) cvc5

Figure 5. Confirmed bugs that affect the corresponding release versions of Z3 and cvc5.

release version against the test instances of confirmed bugs reported by Once4All for Z3 and cvc5, determining the persistence of these bugs across versions. A bug is considered to affect a version if the original formula successfully triggers the bug. Figure 5 illustrates the distribution of confirmed bugs affecting the solvers' release versions. Our analysis reveals that most of the bugs were only present in the trunk versions of the solvers. Specifically, three of the bugs in Z3 remained latent for over six years, indicating that these bugs were not exposed by the developers or other SMT testing tools. In summary, the results suggest that Once4All is capable of identifying long-standing bugs in SMT solvers but is particularly effective in uncovering newly introduced bugs that may result from the ongoing development of solvers. This underscores the practical significance of our approach.

Developers' Feedback. Several issues reported by Once4All have sparked active discussions among solver developers and received positive recognition. For instance, one developer gratefully acknowledged, "*Thank you for this report! This was a new bug that I introduced in my last patch (whoops!)*" Such comments highlight the immediate impact of our findings. Another developer commendation, stating, "*Credits to the reporter for finding the problem*," further underscores the value developers place on these reports. Besides, some filed issues were labeled as *major*[3], reflecting its critical nature.

Finding 1

Once4All is effective in detecting real bugs in SMT solvers, including long-latent bugs as well as bugs on newly added and solver-specific theories.

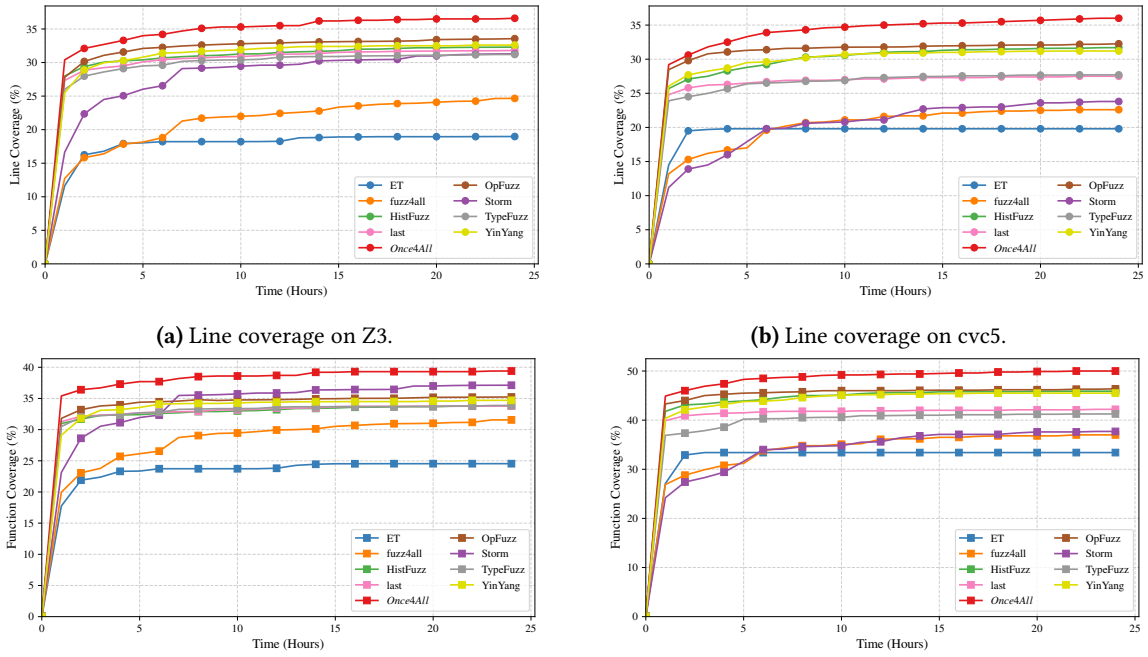
4.3 RQ2: Comparison with Existing Fuzzers

To comprehensively evaluate the effectiveness of Once4All, we compare it with the state-of-the-art fuzzers, including STORM, YinYang, OpFuzz, TypeFuzz, HistFuzz, and Fuzz4All. They are all open-source fuzzers proposed in recent years, capable of supporting multiple logics and identifying a significant number of bugs within SMT solvers. We use the latest versions of these fuzzers provided by their developers in our experiments. Regarding the targeted solvers, we utilize the latest release versions of Z3 (4.14.0) and cvc5 (1.2.1). In this RQ, we investigate the achieved code coverage and the number of unique bugs detected by different fuzzers.

Code Coverage. To evaluate the effectiveness of Once4All in exploring SMT solver implementations, we measure the code coverage it achieves in comparison with baseline fuzzers on Z3 and cvc5. Each fuzzer is executed for 24 hours, and coverage growth is recorded at one-hour intervals. This experimental duration and measurement setup follow established practice in prior work [34]. To ensure a fair comparison, all fuzzers are initialized with the same seed inputs used in *RQ1*. Likewise, both solvers and fuzzers are executed in their default configurations, without enabling optional features or performing parameter tuning. We measure code coverage using `gcov`[4], which tracks the executed source lines and functions in each solver during fuzzing. Figure 6 present the growth of line and function coverage on Z3 and cvc5, respectively, over the 24-hour period. Overall, Once4All consistently outperforms the baselines on both solvers. Across both solvers, all fuzzers show a steady increase in coverage over time and approach a saturation point after approximately 20 hours. Throughout the entire execution, however, Once4All consistently achieves higher coverage than the baseline approaches at each measurement interval. Overall coverage remains below 50% in most cases, with only function coverage on cvc5 slightly exceeding this threshold. This result is expected under our experimental design and consistent with prior findings in testing complex system software. Similar to compiler testing, achieving high absolute code coverage in SMT solvers is inherently difficult: even state-of-the-art compiler testing techniques report line coverage below 40% [25]. Moreover, using default solver configurations to reflect realistic usage leaves substantial functionality disabled and large portions of the codebase unreachable by design. Consequently, code coverage for SMT solvers is generally lower

[3] https://github.com/cvc5/cvc5/pull/10150

[4] https://gcc.gnu.org/onlinedocs/gcc/Gcov.html



(a) Line coverage on Z3.

(b) Line coverage on cvc5.

(c) Function coverage on Z3.

(d) Function coverage on cvc5.

Figure 6. Code coverage growth over time for different fuzzers on Z3 and cvc5.

than that reported for fuzzers targeting applications or libraries, where coverage can exceed 60% [47].

To further examine the differences among fuzzers, we manually analyze the final coverage results after 24 hours. We find that Once4All is complementary to existing fuzzers rather than fully subsuming them. Specifically, while it does not cover all code regions reached by other fuzzers, Once4All exercises additional parts of the solvers that none of the baseline fuzzers reach. For example, on cvc5, Once4All covers solver-specific theory implementations such as the files under the `src/theory/sets/` directory, which implement the theory of sets. However, these files are not reached by any of the baseline fuzzers. In summary, these results show that Once4All achieves higher code coverage than existing SMT solver fuzzing techniques. By exercising a broader portion of the solvers' codebases, Once4All enables more comprehensive exploration of solver behavior.

Bug-Finding Capability. We run Once4All for 24 hours and record the number of unique known bugs found by each fuzzer, consistent with prior work [39]. The seeds employed and the baseline techniques mirror those utilized in the prior code coverage experiments. To identify the unique bugs, we employed the *Correcting Commit* approach [7, 40]. Specifically, we feed a potential bug-triggering instance generated by a fuzzer to different commit versions of the solvers and check whether the bug was fixed in one commit. If a bug can be triggered before a commit while cannot be triggered

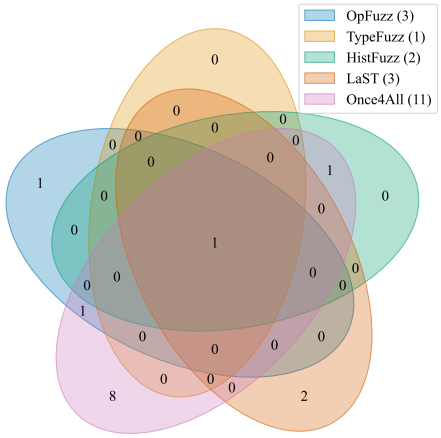


Figure 7. The distribution of unique known bugs detected by Once4All and the baselines on previous versions of solvers.

after the commit, we consider this commit as the correcting commit of the bug. The bugs corresponding to different correcting commits are considered as different bugs. In practice, we exploit binary search to accelerate the process of identifying correcting commits. Note that we focus on using fuzzing tools to find known bugs that have already been resolved, rather than finding new bugs in this experiment, which enables us to conveniently determine the number of unique bugs. The findings, presented in Figure 7, reveal that Once4All excels in detecting the highest number of

unique bugs across the solvers. Previous techniques are incapable of identifying more than three bugs, leading us to infer that these methods encounter difficulties in detecting newly introduced bugs in recent solver versions.

In conclusion, the experiments highlight Once4All's advantage in both bug detection and code coverage efficacy.

Finding 2

Once4All outperforms the state-of-the-art fuzzing techniques in terms of bug-finding capability and code coverage. Specifically, Once4All achieves better performance on cvc5, which is supposed to stem from the fact that cvc5 has more extended theories.

4.4 RQ3: Sensitivity Analysis

This research question examines how individual design choices in Once4All affect its effectiveness. In particular, we focus on two factors: the role of skeleton-guided synthesis and the choice of the underlying LLM. Specifically, to quantify the contribution of skeleton-guided synthesis, we construct a variant of Once4All, denoted as Once4All$_{w/oS}$ (*without Skeletons*). This variant generates test formulas solely using LLM-synthesized generators and does not incorporate skeletons extracted from existing formulas. By comparing Once4All with Once4All$_{w/oS}$, we isolate the impact of skeleton guidance and establish a baseline that reflects the effectiveness of the LLM component alone.

In addition to the synthesis strategy, we investigate whether Once4All's performance is sensitive to the choice of LLM. To this end, we create two additional variants of Once4All, each using a different LLM for grammar summarization and generator synthesis:

- Once4All$_{Gemini}$, which uses Gemini 2.5 Pro [42];
- Once4All$_{Claude}$, which uses Claude 4.5 sonnet [1].

All variants are evaluated under the same experimental setup as in *RQ2*. We measure both code coverage and bug-finding effectiveness to assess how these design choices influence Once4All's overall performance. Figure 8 shows the growth of code coverage over time for Once4All and its variants on Z3 and cvc5. Removing skeleton guidance leads to a clear degradation in coverage: Once4All$_{w/oS}$ consistently explores fewer code paths on both solvers. In contrast, the variants using alternative LLMs, Once4All$_{Gemini}$ and Once4All$_{Claude}$, achieve coverage comparable to the original Once4All, with only minor differences. The reduced coverage of Once4All$_{w/oS}$ also translates into weaker bug-finding performance. As shown in Figure 9, this variant detects only a subset of the bugs uncovered by Once4All. Although Once4All$_{w/oS}$ remains effective to some extent, skeleton guidance substantially improves the ability to exercise deeper solver behaviors and expose additional bugs. Meanwhile, both Once4All$_{Gemini}$ and Once4All$_{Claude}$ identify a similar number of bugs as Once4All, indicating that the framework generalizes well across different LLMs.

Finding 3

Skeleton guidance plays a key role in Once4All's effectiveness, significantly improving both code coverage and bug discovery. Using different LLMs yields comparable results, demonstrating that Once4All's design is robust to the choice of LLM.

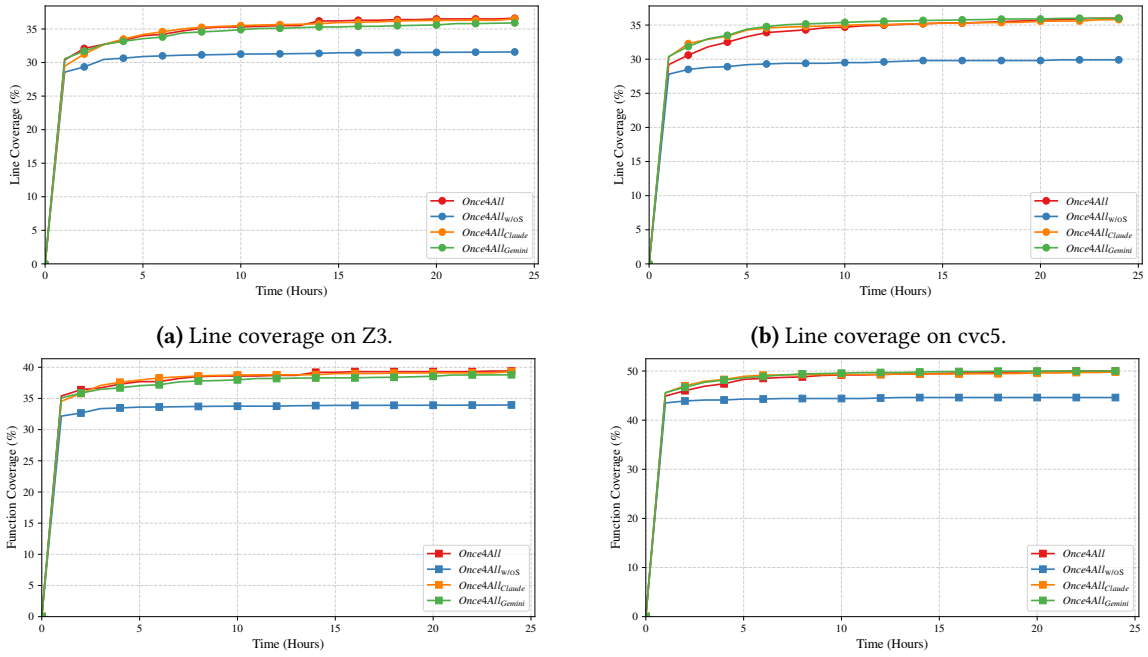
4.5 Bug Sample

To demonstrate Once4All's effectiveness in uncovering bugs across both standard and newly extended theories, we present several representative case studies.

Figure 10a shows a bug in cvc5's finite field theory, which is an extended theory introduced in 2022. In this case, the `ff.bitsum` operator incorrectly ignored coefficient multipliers for constant children. As explained by the developer, the constraint encodes the equation $v = v^2 + 2 \mod 3$ using `ff.bitsum`, expecting solutions $v = 1$ and $v = 2$ in $\mathbb{F}_3$. However, due to a faulty implementation that added constants without scaling them, the solver misinterpreted the formula as $v = v^2 + 1$, leading to wrong models. The root cause was a missing multiplication by the coefficient in the accumulation logic. The patch fixed this by ensuring proper weighting of constant terms. This case highlights how errors can silently compromise solver correctness, especially in extended theories that are not sufficiently exercised.

Figure 10b exposes a bug in cvc5's set theory, which is also an extended theory specifically customized in cvc5. In this case, the solver failed to reject invalid join operations over nullary relations—sets of `UnitTuple`, which have arity zero. The SMT input attempts to join two such sets, expecting a type error because relational joins are undefined for nullary inputs. Instead, the type-checker assumed non-empty tuple types, resulting in misleading behavior. The fix added an explicit check in the type computation logic to detect empty tuple types and return an informative error message: `"Join requires non-nullary relations"`. This patch aligns the implementation with relational semantics and improves diagnostics for users. The test demonstrates how corner cases in relational encodings, like nullary joins, can reveal overlooked semantic constraints.

Figure 10c presents a bug in Z3's model evaluation logic, where dereferencing a null pointer led to a crash when evaluating partially interpreted functions without default expressions. The crafted input includes divisions by zero and function applications that trigger the creation of partial function interpretations with entries but no associated body. The evaluator assumed the presence of an interpretation term



(a) Line coverage on Z3.

(b) Line coverage on cvc5.

(c) Function coverage on Z3.

(d) Function coverage on cvc5.

Figure 8. Code coverage growth over time for Once4All and its variants on Z3 and cvc5.

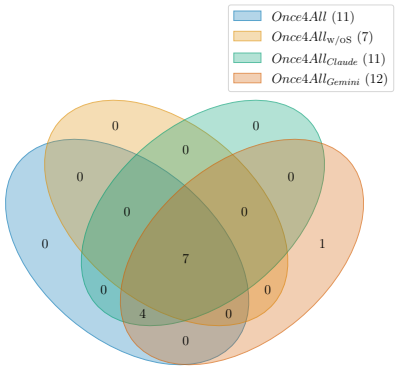


Figure 9. The distribution of unique known bugs detected by Once4All and its variants on Z3 and cvc5.

and attempted substitution without checking for null, resulting in a segmentation fault. The patch introduced a guard that skips substitution if the interpretation is null, safely falling back to alternative handling. This case illustrates how corner-case arithmetic and partial models can interact to produce undefined internal states, emphasizing the importance of defensive programming in model evaluation.

5 Discussion

5.1 Impact of the Self-Correction Mechanism

Once4All incorporates a self-correction mechanism to ensure the validity of generated formulas. This mechanism serves several crucial purposes: it enhances the quality of our formula generators, mitigates the *hallucination* tendency of LLMs which can produce nonsensical outputs, and ultimately boosts the overall effectiveness of Once4All. To assess the impact of the correction mechanism, we measured the proportion of syntactically valid formulas generated before and after its application. Initially, generators derived directly from summarized context-free grammars often produced a high number of invalid formulas that could not be successfully parsed by either cvc5 or Z3. For instance, in syntactically intricate theories like finite fields and newly introduced higher-order theories, the proportion of valid formulas was often below 30%. In contrast, theories like real arithmetic fared much better, with over 90% valid formulas. Our correction mechanism substantially improved these outcomes. For most theories, the proportion of valid formulas increased to nearly 100%. Even in the more challenging and recently added theories, validity improved markedly to over 80%. These results demonstrate that the self-correction mechanism plays a critical role in increasing formula validity, reducing the number of malformed inputs that could otherwise impede bug discovery. Furthermore, Once4All significantly outperforms the direct use of LLMs for test input generation. As shown in prior work [51], relying solely on LLMs can lead to over 50% of formulas being syntactically invalid. In contrast, our approach, with the correction mechanism in place, yields far more useful test inputs.

```
(set-logic QF_FF)
(declare-const v (_ FiniteField 3))
(assert (= v (ff.bitsum (ff.mul v v)
        (as ff-1 (_ FiniteField 3)))))
(check-sat)
```

(a) Invalid model issue regarding Finite Field theory in cvc5. https://github.com/cvc5/cvc5/issues/11969

```
(declare-fun s () (Set UnitTuple))
(assert (rel.join s (as set.empty (Set
    UnitTuple))))
```

(b) Segmentation fault issue regarding Set theory in cvc5. https://github.com/cvc5/cvc5/issues/11903

```
(declare-const x15 Bool)
(declare-const x Real)
(declare-const x1 Real)
(declare-const x9 Bool)
(declare-fun v () Real)
(assert (forall ((r Real)) (or x9
(or (= (+ r 1.0) (mod 0 (to_int x)))))))
(assert (and (> 0.0 x1)
(< x (/ 1.0 (* v x))) (<= 0.0 (/ 0 v))))
(check-sat)
```

(c) Segmentation fault issue in terms of standard theory in Z3. https://github.com/Z3Prover/z3/issues/6935

Figure 10. Selected bug samples in Z3 and cvc5.

5.2 Threats to Validity

This study is susceptible to several primary threats that warrant consideration.

Internal. The main internal threat is that the presence of randomness in the fuzzing process could introduce variability in the results of our experiments. To minimize the impact of non-deterministic behavior, we followed established experimental protocols from prior literature [34, 39]. Specifically, we employed long execution windows for code coverage collection to ensure that our results represent stable trends. Another concern is that our evaluation primarily focused on default solver configurations. Because many internal code paths are disabled or inaccessible via the command-line interface under default settings, our results may not reflect the absolute maximum coverage each fuzzer could achieve. However, because our main objective is to perform a fair, head-to-head comparison between tools rather than to maximize absolute coverage, this setup provides a consistent baseline for our observations.

External. A primary external threat is the potential for the LLM to generate inaccurate outputs, commonly referred to as "hallucinations", when summarizing context-free grammars or implementing generators. Such inaccuracies may result in invalid or ineffective test formulas. To address this issue, we integrate a self-correction mechanism into Once4All, which enhances the validity of the generated formulas.

5.3 Limitation and Future Work

Once4All has several limitations that suggest directions for future work. First, Once4All requires synthesized generators to produce Boolean terms. This simplifies mutation and ensures well-typed SMT-LIB formulas, but may limit the diversity of generated inputs. Extending Once4All to support other term types (e.g., bit-vectors, reals, arrays) or mixed-sorts expressions could broaden the search space and uncover additional classes of solver bugs. Second, the current self-correction mechanism emphasizes syntactic validity. It improves conformance to inferred grammars but does not explicitly promote semantic diversity or bug-finding effectiveness. Incorporating solver-driven signals, such as coverage feedback, exploration of rare configurations, or divergent solver behaviors, could guide generators toward more meaningful and stress-inducing formulas. Third, Once4All relies on solver documentation to summarize grammars, but such documentation is often incomplete or informal. This can introduce inaccuracies that reduce generator quality. Future work could draw on alternative sources, including solver codebases, regression tests, SMT-LIB benchmarks, or grammar information inferred directly from solver behavior. Finally, our evaluation focuses on SMT solvers and several advanced LLMs. As new models emerge, it will be valuable to study how model choice and prompting strategies affect validity, diversity, and bug discovery. More broadly, the methodology behind Once4All may extend to other domains that use grammar-based or template-driven testing, such as compilers and interpreters. In addition, we plan to further automate the current semi-automatic bug triage process. In particular, we aim to investigate automated identification of bug-inducing commits and explore how LLMs can assist in diagnosing recurring or similar issues.

6 Related Work

SMT Solver Fuzzing. SMT solvers are widely adopted in modern systems for verification, testing, and synthesis across the systems and programming languages communities [15–17, 19, 28, 31, 37]. Given their widespread adoption and critical role, ensuring the correctness and robustness of SMT solvers is essential, which has motivated extensive research on solver testing and fuzzing. To improve their reliability,

grammar-based fuzzers such as ET [48] and Murxla [32] were introduced, but both rely highly on expert-crafted generation rules. Mutation-based approaches [29, 35, 49, 50] provide an alternative. For instance, OpFuzz [49] mutates the operators in seeds to generate new test formulas. HistFuzz [40] proposes to exploit skeletons of historical bug formulas to produce mutants. However, these methods remain constrained by manual mutation strategies. Our work instead leverages LLMs to synthesize reusable term generators across different theories, including newly updated ones. These generators aim to fill skeletons with diverse terms to produce effective test cases. Our tool, Once4All, has demonstrated effectiveness on state-of-the-art solvers across both standard and solver-specific theories.

Large Language Models. LLMs have achieved remarkable success in generation and reasoning [43], and have been applied to software engineering for code generation, completion, and testing [12, 13, 39, 45, 51, 54]. For example, Ou et al. [34] fuzz compilers with LLM-designed mutators, and Wang et al. [44] further investigate this paradigm. LLMs have also been applied to SMT solver fuzzing [39, 51], such as LaST [39], which retrains LLMs for formula generation. However, direct generation often yields invalid inputs and requires costly retraining or reasoning. Concurrent work, G^{2}FUZZ [55], also integrates LLMs into fuzzing but differs substantially from our approach. It prompts LLMs to produce deterministic scripts that generate a small set of fixed inputs and relies on AFL++ for mutation. Such mutation is not well suited to the highly structured nature of SMT formulas. Moreover, because each generator produces only a handful of inputs, G^{2}FUZZ repeatedly queries the LLM to obtain new generators, incurring recurring latency and nontrivial cost. In contrast, Once4Allqueries the LLM once per theory to synthesize reusable random generators grounded in solver documentation. These generators systematically complete skeletons, avoid repeated LLM interaction, and reduce invalid cases, thereby improving the overall efficiency and scalability of SMT solver fuzzing.

Skeleton-based Testing. Skeleton- or template-based testing is effective in exposing deep bugs. For example, Skeletal Program Enumeration (SPE) [52, 56] tests compilers via systematic skeleton enumeration, while later work applied templates to MLIR compilers [46] and JVMs [24]. These techniques, however, require manually designed strategies or skeletons. Our method reduces this effort by automatically completing skeletons with LLM-synthesized generators. This strategy has already uncovered 43 confirmed bugs in solvers and shows promise for adoption in other domains.

7 Conclusion

This paper introduces Once4All, an innovative approach for testing SMT solvers that leverages the capabilities of LLMs. Instead of directly using LLMs, we employ them to design and implement generators for various SMT theories, including newly added and solver-specific ones. Furthermore, Once4All synthesizes the terms generated by LLM-produced generators into skeletons extracted from real-world formulas. We implemented Once4All as a practical fuzzing tool and conducted a comprehensive evaluation using two well-known SMT solvers, namely Z3 and cvc5. Notably, Once4All has identified 43 confirmed real bugs in the solvers, with 40 of them already fixed by developers. Moreover, our experiments demonstrate that Once4All outperforms state-of-the-art SMT solver fuzzers in terms of code coverage and bug-finding ability. This study demonstrates an effective approach to utilizing LLMs for software system testing, particularly for rapidly evolving systems.

Acknowledgments

We are grateful to the anonymous reviewers for their valuable and constructive feedback. We also express our sincere gratitude to our shepherd, Siva Kesava Reddy Kakarla, for his insightful guidance and support during the revision of this work. We also greatly appreciate the Z3 and cvc5 development teams, in particular Nikolaj Bjørner, Andrew Reynolds, and Clark Barrett, for their support and for carefully examining the bugs we reported. This work was supported in part by the National Natural Science Foundation of China (Grants 624B2067, 62572226, and 62472215), the Jiangsu Natural Science Foundation (Grant BK20231402), the Collaborative Innovation Center of Novel Software Technology and Industrialization.

A Artifact Appendix

A.1 Abstract

This artifact provides the source code, datasets, and experimental scripts for Once4All, a fuzzing framework for SMT solvers presented in our ASPLOS'26 paper. The artifact is designed to reproduce the key results reported in the paper, including code coverage improvements and bug-finding capabilities. Specifically, the package includes: (a) The complete source code of Once4All implemented in Python; (b) The set of LLM-synthesized term generators for various SMT theories (including Z3 and cvc5 specific extensions); (c) The dataset of seed formulas, i.e., the historical bug-triggering formulas used in our evaluation; (d) Scripts to automatically run the experiments such as detecting bugs; (e) A Docker image with the environment pre-configured, including the target solvers (Z3 and cvc5) and necessary tools (e.g., gcov).

A.2 Artifact check-list (meta-information)

- **Program:** Once4All Z3, cvc5
- **Data set:** historical bug seeds
- **Run-time environment:** Linux (Ubuntu 20.04 via Docker)
- **Hardware:** Standard X86_64 machine
- **How much disk space required (approximately)?:** 30GB

- **How much time is needed to prepare workflow (approximately)?:** 15 minutes to download and import the Docker image.
- **How much time is needed to complete experiments (approximately)?:**
  - Minimal "kick-the-tires" run: ~0.5 hour.
  - Full reproduction: ~10 hours.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT License
- **Archived (provide DOI)?:** Yes

A.3 Description

The artifact can be downloaded from the following link: https://doi.org/10.5281/zenodo.17946453

A.3.1 Hardware dependencies. A standard X86_64 machine supporting Docker. Multi-core CPUs are recommended to parallelize solver execution.

A.3.2 Software dependencies.

- Docker
- (Optional) OpenAI API Key: Required only if the user wishes to re-run the "Generator Construction" phase. The artifact includes pre-synthesized generators, so an API key is **not** required for standard reproduction.

A.4 Installation

Download the artifact package and load the Docker image:

```
$ gunzip -c once4all.tar.gz > once4all.tar
$ cat once4all.tar | docker import - once4all
```

A.5 Experiment Workflow

The artifact supports the reproduction of the main experimental results discussed in Section 4 of the paper. The overall workflow is summarized as follows:

1. **Read the documentation.** Documentation included in the artifact describes the directory structure, available scripts, and expected outputs.
2. **Start the Docker container.** A pre-configured Docker image is provided through the Zenodo archive, containing all required dependencies and solver binaries. Instructions for launching the container are included in the documentation.
3. **Inspect reported bugs.** The artifact includes the links to bugs identified by Once4All.
4. **Reproduce experimental results.** The provided scripts enable the reproduction of the primary experimental results reported in the evaluation section.

A.6 Evaluation and Expected Results

The artifact provides the necessary datasets and scripts to reproduce the empirical evaluation results presented in Section 4. In particular, the results reported in the main experiments can be regenerated using the provided resources. Other findings discussed in the paper are based on qualitative analysis and manual inspection and are therefore not fully automated. Additional details are available in the accompanying documentation and artifact archive.

References

[1] Anthropic. 2025. Introducing Claude Sonnet 4.5. https://www.anthropic.com/news/claude-sonnet-4-5 Accessed: 2025-12-23.

[2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2022. *The satisfiability modulo theories library (SMT-LIB).* Retrieved 2022-08-18 from http://smtlib.cs.uiowa.edu/

[3] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2025. *The SMT-LIB Standard: Version 2.7.* Technical Report. Department of Computer Science, The University of Iowa. Available at www.SMT-LIB.org.

[4] Clark Barrett and Cesare Tinelli. 2018. Satisfiability modulo theories. In *Handbook of model checking*. Springer, 305–343.

[5] Robert Brummayer and Armin Biere. 2009. Fuzzing and Delta-Debugging SMT Solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories* (Montreal, Canada) *(SMT '09)*. Association for Computing Machinery, New York, NY, USA, 1–5. doi:10.1145/1670412.1670413

[6] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.. In *OSDI*, Vol. 8. 209–224.

[7] Junjie Chen, Wenxiang Hu, Dan Hao, Yingfei Xiong, Hongyu Zhang, Lu Zhang, and Bing Xie. 2016. An empirical comparison of compiler testing techniques. In *Proceedings of the 38th International Conference on Software Engineering, ICSE 2016, Austin, TX, USA, May 14-22, 2016*, Laura K. Dillon, Willem Visser, and Laurie A. Williams (Eds.). ACM, 180–190. doi:10.1145/2884781.2884878

[8] The International SMT Competition. 2020. *SMT-COMP.* Retrieved 2020-05-15 from https://smt-comp.github.io/2019/index.html

[9] cvc5. 2022. *cvc5 fuzzing guidelines.* Retrieved 2022-08-18 from https://github.com/cvc5/cvc5/wiki/Fuzzing-cvc5

[10] cvc5. 2025. *Non-standard or extended theories in cvc5.* Retrieved 2024-12-1 from https://cvc5.github.io/docs-ci/docs-main/theories/theories.html#non-standard-or-extended-theories

[11] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.

[12] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2022. Fuzzing Deep-Learning Libraries via Large Language Models. *CoRR* abs/2212.14834 (2022). arXiv:2212.14834 doi:10.48550/arXiv.2212.14834

[13] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large Language Models are Edge-Case Fuzzers: Testing Deep Learning Libraries via FuzzGPT. *CoRR* abs/2304.02014 (2023). arXiv:2304.02014 doi:10.48550/arXiv.2304.02014

[14] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin T. Vechev. 2018. NetComplete: Practical Network-Wide Configuration Synthesis with Autocompletion. In *15th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2018, Renton, WA, USA, April 9-11, 2018*, Sujata Banerjee and Srinivasan Seshan (Eds.). USENIX Association, 579–594. https://www.usenix.org/conference/nsdi18/presentation/el-hassany

[15] Rongxin Han, Jingyu Wang, Qi Qi, Haifeng Sun, Chaowei Xu, Zhaoyang Wan, Zirui Zhuang, Yichuan Yu, and Jianxin Liao. 2024. NetRen: Service Migration-Driven Network Renascence with Synthesizing Updated Configuration. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla,*

CA, USA, 27 April 2024- 1 May 2024, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 708–721. doi:10.1145/3620666.3651365

[16] Travis Hance, Andrea Lattuada, Chris Hawblitzel, Jon Howell, Rob Johnson, and Bryan Parno. 2020. Storage Systems are Distributed Systems (So Verify Them That Way!). In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 99–115. https://www.usenix.org/conference/osdi20/presentation/hance

[17] Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Arjun Narayan, Bryan Parno, Danfeng Zhang, and Brian Zill. 2014. Ironclad Apps: End-to-End Security via Automated Full-System Verification. In *11th USENIX Symposium on Operating Systems Design and Implementation, OSDI '14, Broomfield, CO, USA, October 6-8, 2014*, Jason Flinn and Hank Levy (Eds.). USENIX Association, 165–181. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/hawblitzel

[18] Heqing Huang. 2023. *CVE-2020-19725*. Retrieved 2023-11-5 from https://nvd.nist.gov/vuln/detail/CVE-2020-19725#

[19] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*, Tim Brecht and Carey Williamson (Eds.). ACM, 47–62. doi:10.1145/3341301.3359630

[20] Siva Kesava Reddy Kakarla, Ryan Beckett, Todd D. Millstein, and George Varghese. 2022. SCALE: Automatically Finding RFC Compliance Bugs in DNS Nameservers. In *19th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2022, Renton, WA, USA, April 4-6, 2022*, Amar Phanishayee and Vyas Sekar (Eds.). USENIX Association, 307–323. https://www.usenix.org/conference/nsdi22/presentation/kakarla

[21] Eunsuk Kang, Stéphane Lafortune, and Stavros Tripakis. 2019. Automated Synthesis of Secure Platform Mappings. In *Computer Aided Verification - 31st International Conference, CAV 2019, New York City, NY, USA, July 15-18, 2019, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 11561)*, Isil Dillig and Serdar Tasiran (Eds.). Springer, 219–237. doi:10.1007/978-3-030-25540-4_12

[22] Jongwook Kim, Sunbeom So, and Hakjoo Oh. 2023. Diver: Oracle-Guided SMT Solver Testing with Unrestricted Random Mutations. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2224–2236. doi:10.1109/ICSE48619.2023.00187

[23] Gereon Kremer, Aina Niemetz, and Mathias Preiner. 2021. ddSMT 2.0: Better Delta Debugging for the SMT-LIBv2 Language and Friends. In *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 12760)*, Alexandra Silva and K. Rustan M. Leino (Eds.). Springer, 231–242. doi:10.1007/978-3-030-81688-9_11

[24] Cong Li, Yanyan Jiang, Chang Xu, and Zhendong Su. 2023. Validating JIT Compilers via Compilation Space Exploration. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace (Eds.). ACM, 66–79. doi:10.1145/3600006.3613140

[25] Shaohua Li, Theodoros Theodoridis, and Zhendong Su. 2024. Boosting Compiler Testing by Injecting Real-World Code. *Proc. ACM Program. Lang.* 8, PLDI (2024), 223–245. doi:10.1145/3656386

[26] Yi Li, Aws Albarghouthi, Zachary Kincaid, Arie Gurfinkel, and Marsha Chechik. 2014. Symbolic optimization with SMT solvers. In *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20-21, 2014*, Suresh Jagannathan and Peter Sewell (Eds.). ACM, 607–618. doi:10.1145/2535838.2535857

[27] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 530–543. doi:10.1145/3575693.3575707

[28] Nuno P. Lopes, Nikolaj S. Bjørner, Patrice Godefroid, Karthick Jayaraman, and George Varghese. 2015. Checking Beliefs in Dynamic Networks. In *12th USENIX Symposium on Networked Systems Design and Implementation, NSDI 15, Oakland, CA, USA, May 4-6, 2015*. USENIX Association, 499–512. https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/lopes

[29] Muhammad Numair Mansur, Maria Christakis, Valentin Wüstholz, and Fuyuan Zhang. 2020. Detecting critical bugs in SMT solvers using blackbox mutational fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 701–712. doi:10.1145/3368089.3409763

[30] MicroSoft. 2025. *Z3 Guide*. Retrieved 2024-12-1 from https://microsoft.github.io/z3guide/

[31] Luke Nelson, Helgi Sigurbjarnarson, Kaiyuan Zhang, Dylan Johnson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Hyperkernel: Push-Button Verification of an OS Kernel. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 252–269. doi:10.1145/3132747.3132748

[32] Aina Niemetz, Mathias Preiner, and Clark W. Barrett. 2022. Murxla: A Modular and Highly Extensible API Fuzzer for SMT Solvers. In *Computer Aided Verification - 34th International Conference, CAV 2022, Haifa, Israel, August 7-10, 2022, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 13372)*, Sharon Shoham and Yakir Vizel (Eds.). Springer, 92–106. doi:10.1007/978-3-031-13188-2_5

[33] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). arXiv:2303.08774 doi:10.48550/ARXIV.2303.08774

[34] Xianfei Ou, Cong Li, Yanyan Jiang, and Chang Xu. 2024. The Mutators Reloaded: Fuzzing Compilers with Large Language Model Generated Mutation Operators. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4, ASPLOS 2024, Hilton La Jolla Torrey Pines, La Jolla, CA, USA, 27 April 2024 - 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 298–312. doi:10.1145/3622781.3674171

[35] Jiwon Park, Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2021. Generative type-aware mutation for testing SMT solvers. *Proc. ACM Program. Lang.* 5, OOPSLA (2021), 1–19. doi:10.1145/3485529

[36] John Regehr, Yang Chen, Pascal Cuoq, Eric Eide, Chucky Ellison, and Xuejun Yang. 2012. Test-case reduction for C compiler bugs. In *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, Jan Vitek, Haibo Lin, and Frank Tip (Eds.). ACM, 335–346. doi:10.1145/2254064.2254104

[37] Rathin Singha, Rajdeep Mondal, Ryan Beckett, Siva Kesava Reddy Kakarla, Todd D. Millstein, and George Varghese. 2024. MESSI: Behavioral Testing of BGP Implementations. In *21st USENIX Symposium on Networked Systems Design and Implementation, NSDI 2024, Santa Clara, CA, April 15-17, 2024*, Laurent Vanbever and Irene Zhang (Eds.). USENIX Association. https://www.usenix.org/conference/nsdi24/presentation/singha

[38] SMT-LIB Initiative. 2025. *News of SMT-LIB*. https://smt-lib.org/news.shtml

[39] Maolin Sun, Yibiao Yang, Yang Wang, Ming Wen, Haoxiang Jia, and Yuming Zhou. 2023. SMT Solver Validation Empowered by Large Pre-Trained Language Models. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1288–1300. doi:10.1109/ASE56229.2023.00180

[40] Maolin Sun, Yibiao Yang, Ming Wen, Yongcong Wang, Yuming Zhou, and Hai Jin. 2023. Validating SMT Solvers via Skeleton Enumeration Empowered by Historical Bug-Triggering Inputs. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 69–81. doi:10.1109/ICSE48619.2023.00018

[41] Alan Tang, Siva Kesava Reddy Kakarla, Ryan Beckett, Ennan Zhai, Matt Brown, Todd D. Millstein, Yuval Tamir, and George Varghese. 2021. Campion: debugging router configuration differences. In *ACM SIGCOMM 2021 Conference, Virtual Event, USA, August 23-27, 2021*, Fernando A. Kuipers and Matthew C. Caesar (Eds.). ACM, 748–761. doi:10.1145/3452296.3472925

[42] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *CoRR* abs/2507.06261 (2025). arXiv:2507.06261 doi:10.48550/ARXIV.2507.06261

[43] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *CoRR* abs/2307.09288 (2023). arXiv:2307.09288 doi:10.48550/arXiv.2307.09288

[44] Bo Wang, Pengyang Wang, Chong Chen, Qi Sun, Jieke Shi, Chengran Yang, Ming Deng, Youfang Lin, Zhou Yang, and David Lo. 2025. Mut4All: Fuzzing Compilers via LLM-Synthesized Mutators Learned from Bug Reports. *CoRR* abs/2507.19275 (2025). arXiv:2507.19275 doi:10.48550/ARXIV.2507.19275

[45] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging Pre-trained Models and Downstream Tasks for Source Code Understanding. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 287–298. doi:10.1145/3510003.3510062

[46] Haoyu Wang, Junjie Chen, Chuyue Xie, Shuang Liu, Zan Wang, Qingchao Shen, and Yingquan Zhao. 2023. MLIRSmith: Random Program Generation for Fuzzing MLIR Compiler Infrastructure. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 1555–1566. doi:10.1109/ASE56229.2023.00120

[47] Yanhao Wang, Xiangkun Jia, Yuwei Liu, Kyle Zeng, Tiffany Bao, Dinghao Wu, and Purui Su. 2020. Not All Coverage Measurements Are Equal: Fuzzing by Coverage Accounting for Input Prioritization. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society. https://www.ndss-symposium.org/ndss-paper/not-all-coverage-measurements-are-equal-fuzzing-by-coverage-accounting-for-input-prioritization/

[48] Dominik Winterer and Zhendong Su. 2024. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. *Proc. ACM Program. Lang.* 8, OOPSLA2 (2024), 2378–2401. doi:10.1145/3689795

[49] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 193:1–193:25. doi:10.1145/3428261

[50] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 718–730. doi:10.1145/3385412.3385985

[51] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 126:1–126:13. doi:10.1145/3597503.3639121

[52] Xinmeng Xia, Yang Feng, Qingkai Shi, James A. Jones, Xiangyu Zhang, and Baowen Xu. 2024. Enumerating Valid Non-Alpha-Equivalent Programs for Interpreter Testing. *ACM Trans. Softw. Eng. Methodol.* (feb 2024). doi:10.1145/3647994 Just Accepted.

[53] Peisen Yao, Heqing Huang, Wensheng Tang, Qingkai Shi, Rongxin Wu, and Charles Zhang. 2021. Skeletal approximation enumeration for SMT solver testing. In *ESEC/FSE '21: 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, August 23-28, 2021*, Diomidis Spinellis, Georgios Gousios, Marsha Chechik, and Massimiliano Di Penta (Eds.). ACM, 1141–1153. doi:10.1145/3468264.3468540

[54] Guixin Ye, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Xiaoyang Sun, Lizhong Bian, Haibo Wang, and Zheng Wang. 2021. Automated conformance testing for JavaScript engines via deep compiler fuzzing. In *PLDI '21: 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, Virtual Event, Canada, June 20-25, 2021*, Stephen N. Freund and Eran Yahav (Eds.). ACM, 435–450. doi:10.1145/3453483.3454054

[55] Kunpeng Zhang, Zongjie Li, Daoyuan Wu, Shuai Wang, and Xin Xia. 2025. Low-Cost and Comprehensive Non-textual Input Fuzzing with LLM-Synthesized Input Generators. In *34th USENIX Security Symposium, USENIX Security 2025, Seattle, WA, USA, August 13-15, 2025*, Lujo Bauer and Giancarlo Pellegrino (Eds.). USENIX Association, 6999–7018. https://www.usenix.org/conference/usenixsecurity25/presentation/zhang-kunpeng

[56] Qirun Zhang, Chengnian Sun, and Zhendong Su. 2017. Skeletal program enumeration for rigorous compiler testing. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18-23, 2017*, Albert Cohen and Martin T. Vechev (Eds.). ACM, 347–361. doi:10.1145/3062341.3062379